Ders 18 Çalışma Özeti

Ders 18 Çalışma Özeti

Genel Konular

- Dersin tarihi 2 Aralık 2020 Çarşamba. Bu, günün ikinci kaydıdır. Ders Zoom üzerinden yapıldı.
  - Kayıt, bir önceki oturumda (ders 17) başlanan 0-1 sırt çantası (0-1 knapsack) probleminin devamıyla açılır.
- 0-1 sırt çantası probleminin dinamik programlama (dynamic programming) ile çözümü
  - İki durum: eşya alınır ya da alınmaz.
  - Başlangıç koşulları ve tek bağıntıda birleştirme.
  - Tablonun doldurulması ve tablodan geriye doğru giderek seçilen eşyaların bulunması (optimal çözümün yeniden kurulması, reconstruction).
- Robot ile madeni para toplama problemi (robot coin collection problem)
  - Robot ızgaranın sol üst köşesinden girer, sağ alt köşeden çıkar. Her adımda yalnız sağa ya da aşağı gider.
  - Amaç: yola çıkmadan en çok parayı toplayacak rotayı belirlemek.
  - Tablonun doldurulması, sözde kod (pseudocode), zaman ve yer karmaşıklığı, rotanın geri izlenmesi.
- Gelecek haftanın konuları.

Hocanın Özellikle Vurguladığı Kısımlar

- Dinamik programlamada her hücre, o ana kadarki alt problemin en iyi sonucudur.
  - Bir önceki satırın en iyi sonucu yukarıdan aşağıya taşınır. Ya da eşya eklenirse, kalan ağırlığın bir önceki satırdaki en iyi değeri eşyanın değerine eklenir.
- Sırt çantasında yalnız en büyük değeri bulmak yetmez. Hangi eşyaların seçildiği de bulunmalıdır.
  - Bunun için sondaki hücreden geriye gidilir. Hücre, bir üstündeki hücreyle karşılaştırılır.
- Robot problemi sırt çantasından farklıdır.
  - Sırt çantasında bazı satırlar atlanabilir. Robot ise her adımda bir komşu hücreye mutlaka uğrar.
  - Geri izlerken hangi tarafa gidileceğini, üstteki ve soldaki değerlerden büyük olanı belirler.
- En iyi rota birden fazla olabilir. Bu durumda geri izlerken iki farklı yol çıkar.
- Dinamik programlama mantığının oturması zaman alır. Bu yüzden uygulama saatlerinde bol örnek çözülüyor.

Kısa Tekrar Notları

- 0-1 sırt çantası bağıntısı ($P[i, w]$: ilk i eşya ve w kapasiteyle elde edilebilecek en büyük değer):
  - $P[0, w] = 0$ ve $P[i, 0] = 0$.
  - $w_i > w$ ise $P[i, w] = P[i - 1, w]$.
  - Aksi hâlde $P[i, w] = \max\{P[i - 1, w - w_i] + v_i,\ P[i - 1, w]\}$.
- Geri izleme: $P[i, w] \neq P[i-1, w]$ ise i. eşya seçilmiştir; $w \leftarrow w - w_i$ yapılıp bir üst satıra çıkılır. Eşitse eşya seçilmemiştir; yalnız bir üst satıra çıkılır.
- Örnekte kapasite 5, en büyük değer 37, seçilen eşyalar 1, 2 ve 4.

- Robot problemi bağıntısı (c_{ij}: hücrede para varsa 1, yoksa 0):
  - $F[1,1] = c_{11}$.
  - $F[1,j] = F[1,j-1] + c_{1j}$ ve $F[i,1] = F[i-1,1] + c_{i1}$.
  - $F[i,j] = \max\{F[i-1,j],\ F[i,j-1]\} + c_{ij}$.
  - Sonuç $F[n,m]$.
- Robot probleminde zaman ve yer karmaşıklığı $\Theta(nm)$. Rotanın geri izlenmesi $\Theta(n+m)$ adım alır.

Detaylı Açıklamalar

0-1 sırt çantası: bağıntının kurulması

Derste kullanılan örnek (kareden doğrulandı): kapasite $W = 5$.

Eşya	Ağırlık w_i	Değer v_i
1	2	12
2	1	10
3	3	20
4	2	15

- **Durum 1, eşya alınır:** Eşyanın değeri eklenir. Ağırlığı kapasiteden düşülür. Bir önceki satırda kalan kapasitenin en iyi değerine bakılır: $P[i-1, w-w_i] + v_i$.
  - Bir öğrenci örneğe özel "$w-2$" dedi. Hoca bunu $w - w_i$ olarak genelleştirdi.
- **Durum 2, eşya alınmaz:** Kapasite değişmez. Bir üst satırın aynı sütunu alınır: $P[i-1, w]$.
- **Durum 3, eşya sığmaz:** $w_i > w$ ise eşyayı alma şansı yoktur. Doğrudan $P[i-1, w]$ yazılır.
- **Başlangıç:** Sıfırıncı satır ve sıfırıncı sütun 0'dır.

Hoca tahtada bağıntıyı şöyle yazdı:

$$P[i,w] = \begin{cases} P[i-1,w] & w_i > w \\ \max\{P[i-1,w-w_i] + v_i,\ P[i-1,w]\} & \text{diğer} \end{cases}$$

0-1 sırt çantası: tablo

Satırlar eşyalar (i), sütunlar kapasite (w). Tablo karedeki değerlerle ve Python ile doğrulandı:

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Derste adım adım hesaplanan hücreler:

- $P[1,1] = 0$: 1. eşyanın ağırlığı 2, kapasite 1. Eşya sığmaz, yukarıdaki 0 taşınır.
- $P[1,2] = 12$: Eşya alınırsa $12 + P[0,0] = 12$. Alınmazsa $P[0,2] = 0$. En büyüğü 12.
- $P[2,3] = 22$: Eşya alınırsa $10 + P[1,2] = 22$. Alınmazsa $P[1,3] = 12$. En büyüğü 22.
- $P[3,5] = 32$: 3. eşya alınırsa $20 + P[2,2] = 20 + 12 = 32$.

0-1 sırt çantası: seçilen eşyaların bulunması

- $P[4,5] = 37$, $P[3,5] = 32$. Eşit değil, 4. eşya alınmış. Kalan kapasite $5 - 2 = 3$.
- $P[3,3] = 22$, $P[2,3] = 22$. Eşit, 3. eşya alınmamış.
- $P[2,3] = 22$, $P[1,3] = 12$. Eşit değil, 2. eşya alınmış. Kalan kapasite $3 - 1 = 2$.

- $P[1,2] = 12$, $P[0,2] = 0$. Eşit değil, 1. eşya alınmış.
- Sonuç: 4, 2 ve 1 numaralı eşyalar. Toplam ağırlık 5, toplam değer 37.

Bir öğrenci önce aynı satırdaki soldaki hücreyle (37 ile 30) karşılaştırmayı önerdi. Hoca doğru karşılaştırmanın bir üstteki hücreyle (37 ile 32) yapıldığını gösterdi.

Robot ile madeni para toplama

Derste kullanılan ızgara 5 satır, 6 sütun. Paralar şu hücrelerde (kareden okundu): (1,5), (2,2), (2,4), (3,4), (3,6), (4,3), (4,6), (5,1), (5,5).

- İlk satır yalnız soldan, ilk sütun yalnız yukarıdan gelebilir.
- Diğer hücrelere ya yukarıdan ya soldan gelinir. Büyük olan seçilir ve hücredeki para eklenir.
- Doldurulan tablo (Python ile doğrulandı):

$i\backslash j$	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	5

- Robot en fazla 5 para toplar.
- Geri izlemede bir noktada iki komşunun değeri eşit çıkar. Bu yüzden iki farklı en iyi rota vardır.
  - Hoca iki rotayı "önce sağa gidip sonra aşağı inip …" ve "önce aşağı sonra sağa …" diye sözlü olarak özetledi; altyazıdaki adım dizileri tam tutarlı değil. Tablo ve Python sonucuna göre iki rota yalnız ilk iki adımda ayrışır.

Hocanın tahtada birlikte yazdırdığı sözde kod:

```
F[1,1] <- C[1,1]
for j <- 2 to m do
    F[1,j] <- F[1,j-1] + C[1,j]
for i <- 2 to n do
    F[i,1] <- F[i-1,1] + C[i,1]
    for j <- 2 to m do
        F[i,j] <- max(F[i-1,j], F[i,j-1]) + C[i,j]
return F[n,m]
```

- Zaman karmaşıklığı: her hücre bir kez hesaplanır, $\Theta(nm)$. Örnekte $5 \times 6 = 30$ hücre.
- Yer karmaşıklığı: tablo $\Theta(nm)$.

Gelecek hafta

- Gelecek hafta: düzenleme uzaklığı (edit distance, Levenshtein), ardından en uzun ortak alt dizi (longest common subsequence). Sonra graflara geçilecek.

Ders sonrası notlar

- Bağıntıdaki sığmama koşulu ağırlıkla ilgilidir: $w_i > w$. Altyazıda "V, I, W" diye geçiyor. Tahtada doğru olarak $w_i > w$ yazılı.
- Hoca bir an 4. eşyanın ağırlığını 15 diye söyledi, sonra düzeltti. 15 değeridir, ağırlığı 2'dir.
- 0-1 sırt çantasının dinamik programlama çözümü $\Theta(nW)$ zaman ve yer kullanır. Geri izleme $O(n)$ adımdır. Bu sınır kapasite W'ye bağlı olduğu için sözde polinomdur (pseudo-polynomial).
- Robot probleminde bir hücreye "yukarıdan ya da soldan" gelme kuralı yalnız sağa ve aşağı hareket serbest olduğu için geçerlidir. Çapraz hareket serbest olsaydı bağıntıya $F[i-1, j-1]$ de eklenirdi.

- Altyazıda “Napsack”, “Nepsek” diye geçen terim knapsack, “remote coin collection” diye geçen terim robot coin collection, “koyun” diye geçen kelime coin (para) anlamındadır.
- Ekran kayıt cihazının saatinde görünen “9 January” tarihi cihaz ayarıdır. Dersin tarihi 2 Aralık 2020’dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.